

Installation

Package npm 'typescript' : <https://www.npmjs.com/package/typescript>

Utilisation de TypeScript

TypeScript est une surcouche à JavaScript permettant de rendre le langage moins permissif et de structurer le code de manière plus logique.

Le typage

Typescript permet le typage de variables.

Cela permet d'attribuer à une variable le type de donnée qu'elle recevra et d'éviter les effets de bord.



```
1  const isDone: boolean = true;
2  const age: number = 30;
3  const username: string = "Nicolas";
4  const numbers: number[] = [1, 2, 3];
5  const nothing: null = null;
6  const notDefined: undefined = undefined;
7  const object: Object = {key: "value"};
```

Les types customs

Il est possible de déclarer ses propres types customs, permettant de forcer une structure à respecter.



```
1  type Point = { x: number; y: number };
2  let point: Point = { x: 10, y: 20 };
```

Les énumérations

Les énumérations sont un moyen de définir un ensemble de valeurs nommées. Elles permettent de donner des noms lisibles à des constantes, plutôt que d'utiliser des nombres ou des chaînes de caractères "magiques".

```
1  enum Direction {
2      Up = "UP",
3      Down = "DOWN",
4      Left = "LEFT",
5      Right = "RIGHT"
6  }
7
8  console.log(Direction.Right); // "RIGHT"
```

Les classes

Une classe est une structure qui permet de définir un modèle pour créer des objets. Une classe est composée d'attributs (ce que la classe possède) et de méthodes (ce que la classe sait faire).

Une classe instancie des objets à l'aide du constructeur.

```
1  class Author {
2      private firstName: string;
3      private lastName: string;
4      private nationality: string;
5
6      public constructor(firstName:string, lastName: string, nationality: string = "unknown") {
7          this.firstName = firstName;
8          this.lastName = lastName;
9          this.nationality = nationality;
10     }
11
12     public getFullName(): string {
13         return `${this.firstName} ${this.lastName}`;
14     }
15 }
```

Les modificateurs d'accès

Les modificateurs d'accès sont des mots clés permettant d'explicitier ce qui est disponible dans la classe ou non.

public : accessible depuis n'importe quel endroit

protected : accessible depuis la classe et ses enfants

private : accessible depuis la classe

Getters et Setters

Les Getters et Setters sont des méthodes publiques permettant d'accéder à des attributs privés d'une classe. Ils permettent de laisser les attributs privés et de décider quels droits les utilisateurs ont dessus, et la manière dont cela est possible.

```
1  class BankAccount {
2      private balance: number;
3
4      constructor(initialBalance: number) {
5          this.balance = initialBalance;
6      }
7
8      public setBalance(balance: number): void {
9          this.balance = balance;
10     }
11
12     public getBalance(): number {
13         return this.balance;
14     }
15 }
16
17 const account = new BankAccount(100);
18 account.setBalance(150);
19 console.log(account.getBalance());
20
```

Attributs et méthodes statiques

Le mot clé `static` sert à définir des propriétés ou des méthodes qui appartiennent à la classe elle-même et non pas aux instances de cette classe.

Sans `static` : chaque objet créé à partir de la classe possède sa propre copie de la propriété ou peut appeler la méthode.

Avec `static` : la propriété ou la méthode est partagée par toutes les instances et on y accède directement via la classe.



```
1  class User {
2      static createFullName(firstName: string, lastName: string): string {
3          return `${firstName} ${lastName}`;
4      }
5  }
6
7  console.log(User.createFullName("John", "Smith")); // ✓
```

L'héritage

L'héritage est un mécanisme qui permet à une classe de réutiliser et étendre les attributs et méthodes d'une autre classe (classe parente).

La classe enfant reprend toutes les méthodes et attributs du parent et ajoute les siennes par dessus. On peut appeler le *constructeur* de la classe parente avec le mot clé `super()`.

Pour rappel, le mot clé *protected* indique qu'un attribut n'est accessible qu'au sein de sa propre classe, et de ses classes enfants.

Chaque classe ne peut hériter que d'une seule autre classe, mais le chaînage d'héritage peut-être infini (LopRabbit pourrait à son tour hériter de Rabbit).

```
1  class Animal {
2      protected name: string;
3
4      public constructor(name: string) {
5          this.name = name;
6      }
7
8      public move(): void {
9          console.log(`${this.name} is moving`);
10     }
11 }
12
13
14 class Rabbit extends Animal {
15     private earLength: number;
16
17     public constructor(name: string, earLength: number) {
18         super(name);
19         this.earLength = earLength;
20     }
21
22     public hop(): void {
23         console.log(`${this.name} with ears ${this.earLength}cm long is hopping`);
24     }
25 }
26
27 const myRabbit = new Rabbit("Bunny", 12);
28 myRabbit.move(); // 'Bunny is moving'
29 myRabbit.hop(); // 'Bunny with ears 12cm long is hopping'
```

La surcharge de méthode

La surcharge de méthode permet à une classe fille de redéfinir une méthode que sa classe mère intègre déjà.

```
1 class Animal {
2     public speak(): void {
3         console.log("An animal is making noise.");
4     }
5 }
6
7 class Dog extends Animal {
8     public override speak(): void {
9         console.log("The dog barks.");
10    }
11 }
12
13 const d = new Dog();
14 d.speak(); // "The dog barks."
```

Le polymorphisme

Le polymorphisme est la capacité d'un objet à se comporter de différentes manières selon le contexte, tout en restant du même type général. Par exemple, un objet fait avec la classe Rabbit pourrait être passé à une fonction qui attend un Animal (car un lapin reste avant tout un animal).

```
1 function makeItMove(animal: Animal): void {
2     animal.move();
3 }
4
5 makeItMove(new Rabbit("Pumpkin", 14)); // Pumpkin is moving
```

Les classes abstraites

Une classe abstraite est une classe qui ne peut pas être instanciée directement, mais qui sert de modèle pour d'autres classes qui en hériteront.

Une classe abstraite peut contenir :

- Des méthodes avec implémentation ;
- Des méthodes abstraites (déclarées mais sans implémentation) que les classes filles doivent obligatoirement implémenter.

```
1  abstract class Animal {
2      public constructor(protected name: string) {}
3
4      public move(): void {
5          console.log(`${this.name} is moving`);
6      }
7
8      abstract makeSound(): void;
9  }
10
11
12  class Rabbit extends Animal {
13      public makeSound(): void {
14          console.log(`${this.name} squeaks!`);
15      }
16
17      public hop(): void {
18          console.log(`${this.name} is hopping`);
19      }
20  }
```

Les interfaces

Une interface est un contrat qui définit la forme d'un objet : quelles propriétés et quelles méthodes une classe doit posséder.

Une classe peut implémenter autant d'interfaces qu'elle le souhaite.

Les interfaces sont des contrats publics permettant d'assurer qu'une classe qui l'utilise aura les attributs et méthodes qu'elle déclare (ils doivent être publics).

```
1 interface Animal {
2     name: string;
3     move(): void;
4 }
5
6 class Rabbit implements Animal {
7     public readonly name: string;
8
9     public constructor(name: string) {
10         this.name = name;
11     }
12
13     public move(): void {
14         console.log(`${this.name} is hopping`);
15     }
16
17     public hop(): void {
18         console.log(`${this.name} is jumping!`);
19     }
20 }
21
```

Les types génériques

Les types génériques sont un concept permettant de réutiliser du code pour différents types tout en conservant la sécurité du typage. Un type générique est donc un type "paramétrable" défini au moment de l'utilisation.

```
1 class Box<T> {
2     private content: T;
3
4     constructor(value: T) {
5         this.content = value;
6     }
7
8     public getContent(): T {
9         return this.content;
10    }
11 }
12
13 const box1 = new Box<string>("Hello");
14 const box2 = new Box<number>(123);
15
16 console.log(box1.getContent()); // "Hello"
17 console.log(box2.getContent()); // 123
```


Les singletons

Les singletons permettent de n'avoir qu'une seule instance d'une classe dans toute l'application.

On utilise private constructor pour empêcher la création d'instances externes et static instance pour stocker l'instance unique.

```
1 class Database {
2     private static instance: Database;
3
4     private constructor() {
5         console.log("Database initialized");
6     }
7
8     public static getInstance(): Database {
9         if (!Database.instance) {
10             Database.instance = new Database();
11         }
12         return Database.instance;
13     }
14 }
15
16
17 const db1 = Database.getInstance();
18 const db2 = Database.getInstance();
19 console.log(db1 === db2); // true
```

La composition

La composition consiste à former des objets complexes en combinant d'autres objets plutôt qu'en héritant d'une classe.

```
1 class Engine {
2     public start(): void {
3         console.log("Engine started");
4     }
5 }
6
7
8 class Car {
9     private engine: Engine;
10
11     public constructor(engine: Engine) {
12         this.engine = engine;
13     }
14
15     public drive(): void {
16         this.engine.start();
17         console.log("Car is driving");
18     }
19 }
20
21
22 const car = new Car(new Engine());
23 car.drive();
24
```

Les bonnes pratiques

Le nesting abusif rend compliqué la lecture et l'ajout de nouvelles fonctionnalités. Il est préférable d'utiliser le système d'early return pour rendre la lecture plus simple.

```
1 function checkUser(user: User): string {
2   if (user) {
3     if (user.isActive) {
4       if (user.role === "admin") {
5         return "Welcome, active admin!";
6       } else {
7         return "Active user but not admin";
8       }
9     } else {
10      return "Inactive user";
11    }
12  } else {
13    return "No user";
14  }
15 }
```

```
1 function checkUser(user: User): string {
2   if (!user) {
3     return "No user";
4   }
5
6   if (!user.isActive) {
7     return "Inactive user";
8   }
9
10  if (user.role !== "admin") {
11    return "Active user but not admin";
12  }
13
14  return "Welcome, active admin!";
15 }
```

Etant donné qu'une condition ne s'exécutera que si le code dans l'instruction if vaut true, il est inutile de complexifier le processus.

```
1 function isSunday(): boolean {
2   const today = new Date();
3
4   if (today.getDay() === 0) {
5     return true;
6   }
7
8   return false;
9 }
10
11
12 function isSunday(): boolean {
13   const today = new Date();
14   return today.getDay() === 0
15 }
```

L'opérateur ternaire permet d'éviter d'avoir des conditions verbeuses pour la simple déclaration d'une variable.



```
1 let tax;
2 if (price < 50) {
3   tax = 0.05;
4 } else {
5   tax = 0.02;
6 }
7
8
9 const tax = (price < 50) ? 0.05 : 0.02;
```

L'opérateur de coalescence permet de donner une valeur par défaut si l'opérande est null ou undefined.



```
1 const age = null;
2 const isAnAdult = age ?? 0 > 18;
```

Le spread operator permet de copier, cloner ou fusionner des objets et des tableaux en "étaillant" leurs valeurs.



```
1 const numbers = [1, 2, 3];
2 const newNumbers = [...numbers, 4]; // [1, 2, 3, 4]
3
4 const word = "hello";
5 const letters = [...word]; // ["h", "e", "l", "l", "o"]
6
7 const user = { name: "Alice", age: 22 };
8 const updatedUser = { ...user, size: 26 }; // { name: 'Alice', age: 22, size: 26 }
```